Reproducibility Evaluation: Declarative Scenario-based Testing with RoadLogic (ICCPS26)

Quickstart

This is the **bare minimum** fastest way to rerun the experiments. For an in-depth installation guide, see the README file.

Prerequisites

- Docker

Reproducing the figures

To reproduce the figures in the paper, just use the Dockerfile. The experiments run for about 10h in total on my Dell Latitude with a 12th Gen Intel(R) Core(TM) i7-1265U. Your mileage may vary (and the computation time with it).

- Inside this folder, run `docker build -t roadlogic-artifact .`
  (For macOS with Apple Silicon: `docker build --platform=linux/amd64 -t roadlogic-artifact .`)
- Run the container. `docker run --rm -it --name roadlogic-artifact roadlogic-artifact`
  (For macOS with Apple Silicon:
  `docker run --platform=linux/amd64 --rm -it --name roadlogic-artifact roadlogic-artifact`)
  Be careful: remove `--rm` if you want the container to persist after usage.
- Run the script to reproduce the Figures from the paper `bash paper/reproduce_experiments.sh`
- This part takes about 10h.
- Find the results under `paper/figures` **inside the container**.
- After completion, copy the generated figures to the host system at `<PATH>` with:
  `docker cp roadlogic-artifact:/artifact/paper/figures <PATH>`

Expected Reproducibility Outcomes

This artifact includes the source code for our tool RoadLogic that comes with the paper "Declarative Scenario-based Testing with RoadLogic" (ICCPS26).

It features the complete tool with an easy-to-use entry-point to reproduce our experiments in the paper in a runnable bash script. Figures 1, 4, 5, 7, 8 and 9 are reproduced in this script. It generates the corresponding pdf/png files in the folder `paper/figures` inside the container. Most plots are identical to the original figures, or similar when some minor stochasticity makes the experiments not perfectly reproducible. The details are below.

Experiment Structure

Typically our experiment folder have this structure:

```
EXP_FOLDER/
  in/
    spec.osc
    rule_injection.lp
  out/
    animations/
    asp/
```

```
  ...
run.sh
animate.sh
```

In run.sh, one of two experiment modes are used: the vanilla (called base in the paper) variant and the partitioning (called refined in the paper) variant. This is the common code that is used across experiments (without the time keeping code, to reduce complexity in this document).

```
run_vanilla_experiment() {
  EXP="$1"; N="$2"

  exp_reset_out "$EXP"

  make generate-osc-scenarios \
    OSC_N="$N" \
    OSC_FILE="$EXP/in/spec.osc" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations" \
    SCENARIOS_FOLDER="$EXP/out" \
    RULE_INJECTION="$EXP/in/rule_injection.lp"

  make execute-all-scenarios \
    SCENARIOS_FOLDER="$EXP/out/cr_scenarios" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations"

  make -j 8 \
    SCENARIOS_FOLDER="$EXP/out/cr_scenarios" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations" \
    compute-selected-metrics-for-all-scenarios

  make generate-summary \
    SCENARIOS_FOLDER="$EXP/out/cr_scenarios" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations" \
    OSC_FILE="$EXP/in/spec.osc"

  python commonroad_extensions/simulation_analysis/similarity.py \
    --plans-folder "$EXP/out/serialized_models" \
    --output-folder "$EXP/out/similarity"
}
```

```
run_variability_experiment() {
  EXP="$1"; N="$2"

  exp_reset_out "$EXP"

  python create_scenarios/create_variability_instances.py \
    --output-folder "$EXP/out/partitions" \
    --n "$N"

  python create_scenarios/variability_experiments.py \
    --output-folder "$EXP/out" \
    --input-folder "$EXP/out/partitions" \
    --osc-file "$EXP/in/spec.osc" \
    --cr-scenario "cr_scenarios/threelanes.xml" \
    --rule-injection "$EXP/in/rule_injection.lp"
```

```
  make execute-all-scenarios \
    SCENARIOS_FOLDER="$EXP/out/cr_scenarios" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations"

  make -j 8 \
    SCENARIOS_FOLDER="$EXP/out/cr_scenarios" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations" \
    compute-selected-metrics-for-all-scenarios

  make generate-summary \
    SCENARIOS_FOLDER="$EXP/out/cr_scenarios" \
    OUTPUT_FOLDER="$EXP/out/cr_simulations" \
    OSC_FILE="$EXP/in/spec.osc" \
    BLUEPRINT_FOLDER="$EXP/out/aux_asp"

  python commonroad_extensions/simulation_analysis/similarity.py \
    --plans-folder "$EXP/out/serialized_models" \
    --output-folder "$EXP/out/similarity"
}
```

Example Usage

For example, running our working example from the paper is as simple as running `tests/overtake/run.sh` It will:

- run the ASP planner in the base configuration, generating asp instance files, plans, plan animations, etc. in `tests/overtake/out`
- simulate in commonroad using Frenetix, generating simulation traces in `tests/overtake/out/cr_simulations`
- optionally we can call `bash tests/overtake/animate.sh` to create gif animations like `tests/overtake/out/cr_simulations/scenario_0001/simulation_00001/simulation.gif`.

The two input files `tests/overtake/in/spec.osc` and `tests/overtake/in/rule_injection.lp` define the specification. Feel free to try different scenarios.

- The `spec.osc` file is the OpenSCENARIO specification.
- The `rule_injection.lp` is an optional way of adding constraints on the ASP level to further refine the output scenarios.

This example is provided for illustration and exploration and is not used to generate any paper figures.

Experiment Suite

The artifact contains the following experiment folders:

Benchmark experiments (Figures 5,6,8)

- `experiments_src/exp_00X` for X = 1..7 (base mode)
- `experiments_src/exp_00X_b` for X = 1..7 (refined mode)

Visualization experiments (Figures 1,4,7,9)

- `paper/01_model_vis/overtake`
- `paper/02_trajectory_vis/01_overtake`
- `paper/02_trajectory_vis/02_fake_overtake`

- `paper/02_trajectory_vis/03_similarity`

Paper Outcomes vs. Artifact Outputs

All functional outcomes are generated by running the single entry-point script:

```
bash paper/reproduce_experiments.sh
```

This script executes all required experiments and plotting steps and writes the final figures to `paper/figures` **inside the Docker container**.

To access them from the host system, either:

- copy them out using `docker cp` (see quickstart), or
- attach to the container and browse the filesystem (i.e. in VS Code).

Each functional outcome corresponds to one paper figure and is mapped below.

- **[F1]**
  Recreates the trajectory visualization outcome shown in Figure 1 of the paper.
  Generated by the trajectory visualization pipeline (`paper/02_trajectory_vis/01_overtake`) and aggregated by `paper/reproduce_experiments.sh` .
  Output: `paper/figures/Fig_1_alt.pdf` .
  Stochasticity note (Figure 1). Due to minor stochasticity in the ASP solver (Clingo), the depicted plan can differ slightly from the paper. A recent bug fix changed the ASP input, affecting solver ordering. The camera-ready version of the paper will update this figure to ensure full reproducibility.
- **[F2]**
  Recreates the model visualization outcome shown in Figure 4 of the paper.
  Generated by the model visualization pipeline (`paper/01_model_vis/overtake`).
  Outputs: `paper/figures/Fig_4a.pdf` , `paper/figures/Fig_4b.pdf` , `paper/figures/Fig_4c.pdf` .
- **[F3]**
  Recreates the benchmark timing outcome shown in Figure 5 of the paper.
  Generated by running the base benchmark via `experiments_src/run_vanilla_experiments.sh` followed by the visualization step `bash experiments_src/vis_all_experiments.sh`
  which is invoked inside `paper/reproduce_experiments.sh` .
  Output: `paper/figures/Fig_5.pdf` .
  Timing variability note (Figure 5).
  The outcome [F3] includes execution time measurements. Absolute runtimes depend on the hardware, operating system, and system load of the machine on which the artifact is executed. Consequently, timing values in Figure 5 may differ from those reported in the paper. In addition, some experiment instances may reach the configured timeout on different systems, which can slightly change the extent of the grey timeout bars in the plot.
- **[F4]**
  Recreates the plan similarity analysis outcome shown in Figure 7 of the paper.
  Generated from similarity matrices computed for benchmark experiments `exp_001` – `exp_007` in both base and refined modes.
  These matrices are produced during the benchmark runs and collected by `paper/reproduce_experiments.sh` .
  Outputs: `paper/figures/Fig_7_*.png` .
- **[F5]**
  Recreates the combined outcome comparison shown in Figure 8 of the paper.

Generated by running the refined benchmark via `experiments_src/run_partitioning_experiments.sh` followed by the visualization step `bash experiments_src/vis_all_experiments.sh`

which is invoked inside `paper/reproduce_experiments.sh`.

Output: `paper/figures/Fig_8.pdf`.

Timing variability note (Figure 8).
The outcome [F5] includes aggregated timing information from the refined benchmark experiments. As with Figure 5, absolute runtimes and timeout behavior depend on the execution environment. When reproduced on a different system, timing values and the precise length of grey timeout bars may vary slightly, while the qualitative trends reported in the paper remain unchanged.

All functional outcomes listed above are produced automatically when running `paper/reproduce_experiments.sh`; no additional manual steps are required.